

```
# Car Classification using Naive Bayes
```

```
# Import libraries
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Create car dataset
```

```
data = {
```

```
    'Engine_CC': [
```

```
        1197, 1197, 1197, 1197,
```

```
        1498, 1498, 1498, 1498,
```

```
        1197, 1197, 1197, 1197,
```

```
        998, 998, 998, 998,
```

```
        1497, 1497, 1497, 1497
```

```
    ],
```

```
    'Mileage': [
```

```
        18, 19, 18, 20,
```

```
        20, 21, 19, 20,
```

```
        17, 18, 16, 17,
```

```
        22, 23, 21, 22,
```

```
        18, 19, 17, 18
```

```
    ],
```

```
    'Price_Lakh': [
```

```
        7, 7.5, 6.8, 7.2,
```

```
        10, 10.5, 9.8, 10.2,
```

```

8, 8.5, 7.8, 8.2,
6, 6.5, 5.8, 6.2,
9, 9.5, 8.8, 9.2
],

'Car_Name': [
'Swift', 'Swift', 'Swift', 'Swift',
'Honda_City', 'Honda_City', 'Honda_City', 'Honda_City',
'Hyundai_i20', 'Hyundai_i20', 'Hyundai_i20', 'Hyundai_i20',
'Alto', 'Alto', 'Alto', 'Alto',
'Hyundai_Verna', 'Hyundai_Verna', 'Hyundai_Verna', 'Hyundai_Verna'
]
}

```

```

# Convert data into DataFrame

```

```

df = pd.DataFrame(data)

```

```

print("Car Dataset:")

```

```

print(df)

```

```

# Separate input and target

```

```

X = df[['Engine_CC', 'Mileage', 'Price_Lakh']]

```

```

y = df['Car_Name']

```

```

# Convert car names into numbers

```

```

car_names = y.unique()

```

```

name_to_number = {name: i for i, name in enumerate(car_names)}

```

```

number_to_name = {i: name for name, i in name_to_number.items()}

```

```

y = y.map(name_to_number)

```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

```
# Create Naive Bayes model
model = GaussianNB()
```

```
# Train model
model.fit(X_train, y_train)
```

```
# Predict
y_pred = model.predict(X_test)
```

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nCar Classification using Naive Bayes")
print("-----")
print("Accuracy:", accuracy)
print("Accuracy (%):", accuracy * 100)
```

```
# Display actual and predicted car names
print("\nActual vs Predicted:")
```

```
for actual, predicted in zip(y_test, y_pred):
    print(
        number_to_name[actual],
```

```

        "->",
        number_to_name[predicted]
    )

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Display confusion matrix
plt.figure(figsize=(8, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=car_names,
    yticklabels=car_names
)

plt.xlabel("Predicted Car")
plt.ylabel("Actual Car")
plt.title("Naive Bayes - Car Classification")
plt.show()

# Classification Report
print("\nClassification Report:")

print(

```

```
classification_report(  
    y_test,  
    y_pred,  
    target_names=car_names,  
    zero_division=0  
)  
)
```

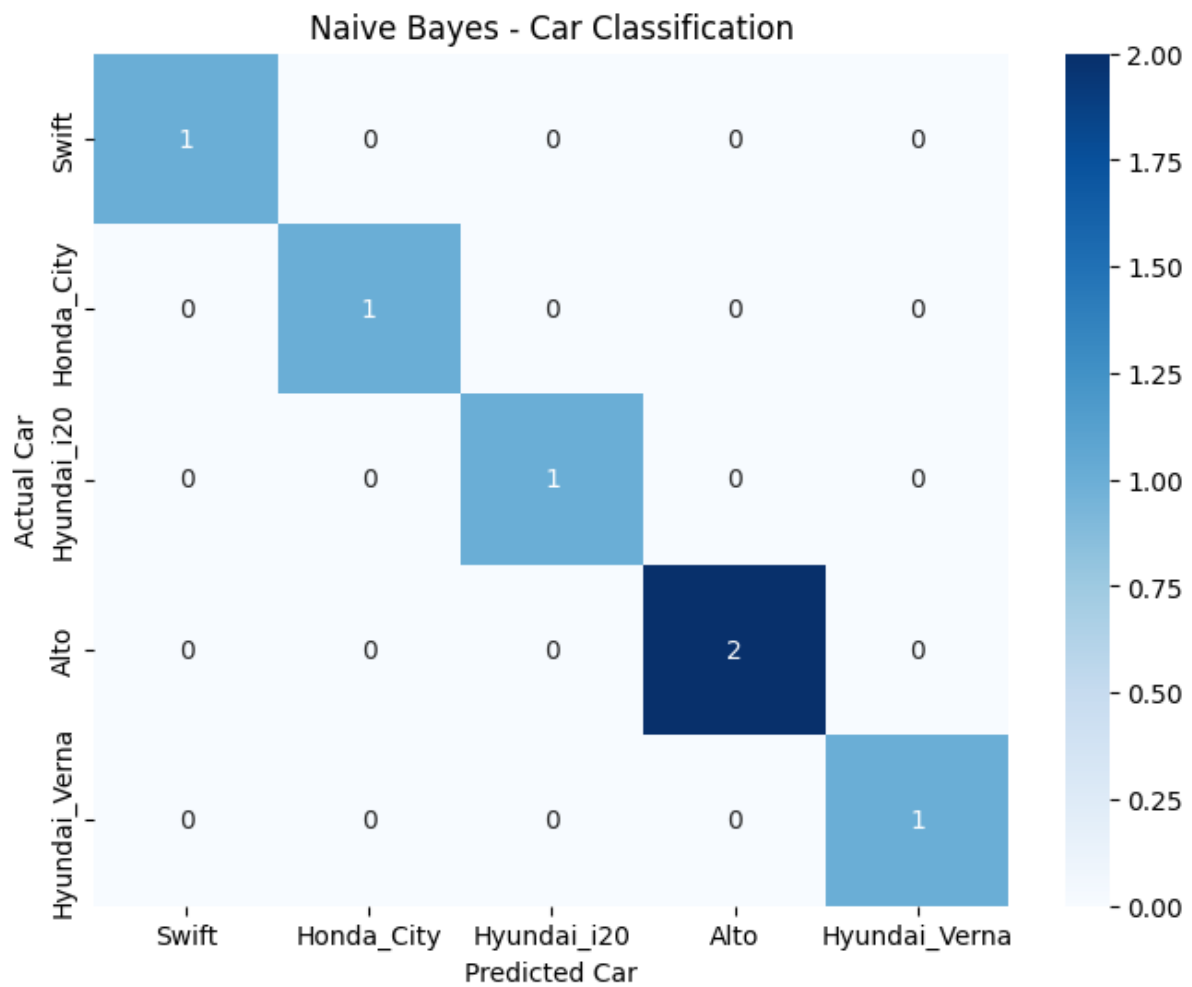
Car Dataset:

	Engine_CC	Mileage	Price_Lakh	Car_Name
0	1197	18	7.0	Swift
1	1197	19	7.5	Swift
2	1197	18	6.8	Swift
3	1197	20	7.2	Swift
4	1498	20	10.0	Honda_City
5	1498	21	10.5	Honda_City
6	1498	19	9.8	Honda_City
7	1498	20	10.2	Honda_City
8	1197	17	8.0	Hyundai_i20
9	1197	18	8.5	Hyundai_i20
10	1197	16	7.8	Hyundai_i20
11	1197	17	8.2	Hyundai_i20
12	998	22	6.0	Alto
13	998	23	6.5	Alto
14	998	21	5.8	Alto
15	998	22	6.2	Alto
16	1497	18	9.0	Hyundai_Verna
17	1497	19	9.5	Hyundai_Verna
18	1497	17	8.8	Hyundai_Verna
19	1497	18	9.2	Hyundai_Verna

Car Classification using Naive Bayes

Accuracy: 1.0

Accuracy (%): 100.0



Classification Report:

	precision	recall	f1-score	support
Swift	1.00	1.00	1.00	1
Honda_City	1.00	1.00	1.00	1
Hyundai_i20	1.00	1.00	1.00	1
Alto	1.00	1.00	1.00	2
Hyundai_Verna	1.00	1.00	1.00	1
accuracy			1.00	6
macro avg	1.00	1.00	1.00	6
weighted avg	1.00	1.00	1.00	6